2. Data Wrangling II Create an “Academic performance” dataset of students and perform the following operations using Python. 1. Scan all variables for missing values and inconsistencies. If there are missing values and/or inconsistencies, use any of the suitable techniques to deal with them. 2. Scan all numeric variables for outliers. If there are outliers, use any of the suitable techniques to deal with them. 3. Apply data transformations on at least one of the variables. The purpose of this transformation should be one of the following reasons: to change the scale for better understanding of the variable, to convert a non-linear relation into a linear one, or to decrease the skewness and convert the distribution into a normal distribution. Reason and document your approach properly.

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import math

import warnings

warnings.filterwarnings("ignore")

%matplotlib inline


rollno = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

name = ["a", "b", "c", "d", "e", "f", "g", "h", "i", "j", np.nan, np.nan, "k", "l", "m"]

marks = [40, 23, 50, 78, 48, 89, 90, 67, 90, 96, 76, np.nan, 97, np.nan, 65]

grade = ["F", "F", "P", "P", "P", "P", "P", "P", "P", "P", "P", "F", "P", np.nan, np.nan]


df = pd.DataFrame({"rollno" : rollno, "name" : name, "marks" : marks, "grade" : grade})


df


df.info()


df.describe()


df.dtypes
```

```
df.columns
```

```
df.isna().sum()
```

```
df["marks"] = df["marks"].fillna(df["marks"].mean())
```

```
def convert_marks(value):
    return int(math.floor(value))
```

```
df["marks"] = df["marks"].apply(convert_marks)
```

```
df = df[df["name"].notna()]
```

```
for index, row in df.iterrows():
    if row["marks"] > 50:
        df.loc[index, "grade"] = "P"
    else:
        df.loc[index, "grade"] = "F"
```

```
df
```

```
df.boxplot(column="marks")
plt.show()
```

```
first_outlier = [16, 'n', 200, 'P']
second_outlier = [17, 'o', -100, 'F']
```

```
df.loc[15] = first_outlier
df.loc[16] = second_outlier
df
```

```
df.boxplot()
plt.show()

sns.countplot(data=df, x=df['marks']);

sns.boxplot(data=df, x='marks');

from matplotlib.cbook import boxplot_stats
outliers = boxplot_stats(df['marks']).pop(0)['fliers']

outliers

df

df = df.drop([15,16], axis=0)

df.boxplot()
plt.show()

from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

df[['marks']] = scaler.fit_transform(df[['marks']])

df

df.boxplot()
plt.show()
```

```
plt.hist(df["marks"])
plt.xlabel("marks")
plt.ylabel("count")
plt.title("marks bins")
```